

Module	ET4047 (Embedded Software)
Date	10/09/2026
Lab No	Lab 2 – Ultrasonic Distance Alarm (Visuino → Wokwi)
Author	Luke Griffin

1 Objectives

- ✓ Use **Visuino** to build a working program from visual components, without hand-writing any code.
- ✓ Read an **HC-SR04** ultrasonic sensor and turn its echo signal into a distance measurement.
- ✓ Use a **Compare** component to trigger an output when the distance crosses a threshold.
- ✓ Generate Arduino source code from a Visuino design and inspect what it produces.
- ✓ Recreate and run the circuit in **Wokwi**, and understand why generated code is not always portable between toolchains.

2 Background

Visuino is a visual programming tool: you drag components onto a canvas, wire their inputs and outputs together, and Visuino generates the Arduino C source for you. **Wokwi** is a browser-based simulator that runs Arduino code against a virtual circuit, so you can test wiring and behaviour with no physical hardware.

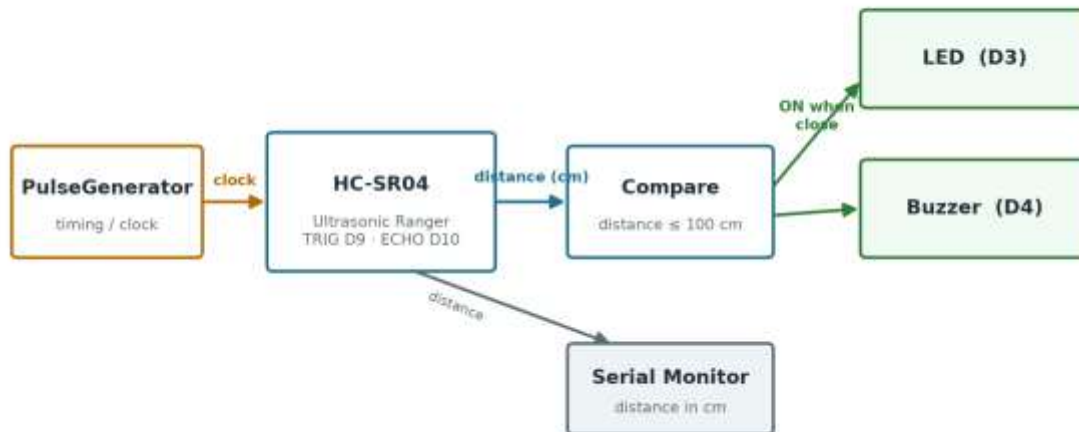
In this lab you will build an **ultrasonic distance alarm**: an HC-SR04 sensor measures how far away the nearest object is, and when that distance drops below a set threshold an LED lights and a buzzer sounds – a simple proximity or reversing alarm.

A note on the toolchain (this is part of the exercise). The code Visuino generates depends on the *Mitov* library (the `OpenWire.h` and `Mitov.h` headers) and on the board core for your target device. Wokwi does not provide that library, so the raw Visuino sketch will not compile in Wokwi as-is. You will therefore build and generate the design in Visuino to learn the visual workflow, then simulate a **functionally-equivalent plain-Arduino sketch** in Wokwi. Recognising this portability gap – and why it exists – is one of the goals of the lab. To keep things consistent, target the classic **Arduino Nano** in both tools.

3 Part A – Build the Alarm in Visuino

Component names vary slightly between Visuino versions – use the search box in the component list to find each part by name.

Part A — Visuino signal flow



Reference – how the Visuino components connect: the ranger measures distance, Compare tests it against 100 cm, and the result drives the LED and buzzer.

1. Open Visuino and select the **Arduino Nano** board as your target.
2. From the component list, add an **Ultrasonic Ranger (HC-SR04)** component to the canvas.
3. Connect the ranger's **Trigger** output to a board digital pin (for example **D9**), and wire the board digital pin for **Echo** (for example **D10**) into the ranger's Echo input.
4. Add a **Compare Analog Value** component. Set its comparison to **Smaller or Equal** and set its value to **100**. This design reports distance in **centimetres** (the ranger's default unit), so **100** means the alarm triggers when an object is within 100 cm.
5. Wire the ranger's **distance output** into the input of the Compare component.
6. Wire the Compare component's **boolean output** to a board digital pin driving the **LED** (for example **D3**). If you are using a buzzer, connect it to a second pin (for example D4) so it sounds on the same signal.
7. *(Optional)* Add a **Serial** element and connect the distance value to it, so the measurement is printed to the Serial Monitor.
8. Generate the code (the **arrow / Build** button, or **F9**). Visuino opens the generated **.ino** sketch in the Arduino IDE.

Capture: a screenshot of your completed Visuino diagram, and a screenshot of the generated code.

[Paste your screenshot here]

Figure 1: Completed Visuino diagram for the ultrasonic alarm

[Paste your screenshot here]

Figure 2: The Arduino code generated by Visuino

Questions?

» Look at the generated code. Which `#include` lines show that it depends on the Mitov library?

» In one sentence, why will this sketch not compile in Wokwi as it stands?

4 Part B – Simulate in Wokwi

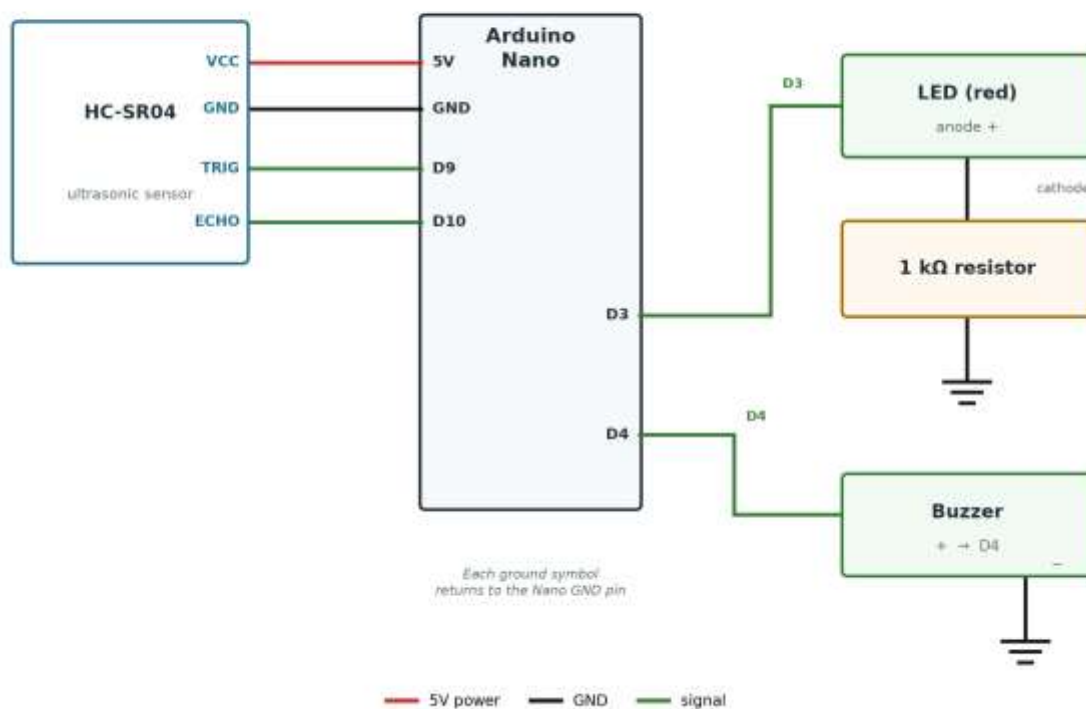
Now recreate the circuit in Wokwi and run the equivalent sketch.

1. Go to Wokwi and start a new project, choosing the **Arduino Nano** template.
<https://wokwi.com>
2. Click the **+** button and add these parts: an **HC-SR04 Ultrasonic Distance Sensor**, an **LED**, a **1 k Ω resistor**, and a **Buzzer**.
3. Wire the circuit according to the table below.

Component / pin	Arduino Nano pin
HC-SR04 VCC	5V
HC-SR04 GND	GND
HC-SR04 TRIG	D9
HC-SR04 ECHO	D10

Component / pin	Arduino Nano pin
LED anode (+, long leg)	D3
LED cathode (–) → 1 kΩ → GND	GND
Buzzer +	D4
Buzzer –	GND

Part B – Wokwi wiring



Reference – how to wire the circuit: HC-SR04 to 5V/GND/D9/D10, the LED anode to D3 through a 1 kΩ resistor to GND, and the buzzer to D4 and GND.

Capture: a screenshot of your completed Wokwi wiring.

[Paste your screenshot here]

Figure 3: Completed Wokwi circuit – Nano, HC-SR04, LED and buzzer

Replace the contents of the Wokwi code editor with the sketch below. This is the plain-Arduino equivalent of your Visuino design: it pulses the trigger, times the echo, converts it to a distance, and drives the LED and buzzer when the object is closer than the threshold.

```
// Lab2 - Ultrasonic distance alarm (Wokwi equivalent of the Visuino design)
// Author: <your name>
```

```
#define TRIG_PIN      9
#define ECHO_PIN      10
#define LED_PIN       3
#define BUZZER_PIN    4
#define THRESHOLD_CM  100    // alarm distance in cm

void setup() {
  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
  pinMode(LED_PIN, OUTPUT);
  pinMode(BUZZER_PIN, OUTPUT);
  Serial.begin(9600);
}

void loop() {
  // 1. send a 10 us trigger pulse
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);
  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);

  // 2. measure the echo pulse width in microseconds
  long duration = pulseIn(ECHO_PIN, HIGH);

  // 3. convert to distance in cm (speed of sound)
  long distance_cm = duration / 58;

  // 4. report the distance
  Serial.print("Distance: ");
  Serial.print(distance_cm);
  Serial.println(" cm");

  // 5. compare against the threshold and drive the outputs
  if (distance_cm > 0 && distance_cm < THRESHOLD_CM) {
    digitalWrite(LED_PIN, HIGH);
    tone(BUZZER_PIN, 1000);    // 1 kHz alarm tone
  } else {
    digitalWrite(LED_PIN, LOW);
    noTone(BUZZER_PIN);
  }

  delay(100);
}
```

4. Press the green **Start Simulation** button.

5. **Click the HC-SR04 sensor** in the canvas while the simulation runs – a distance slider appears. Drag it to change the measured distance.
6. Bring the distance below 100 cm and confirm the **LED lights** and the **buzzer sounds**; move it above 100 cm and confirm they switch off. Open the **Serial Monitor** to watch the distance readings.

Capture: the simulation with the alarm triggered (distance below the threshold), and the Serial Monitor output.

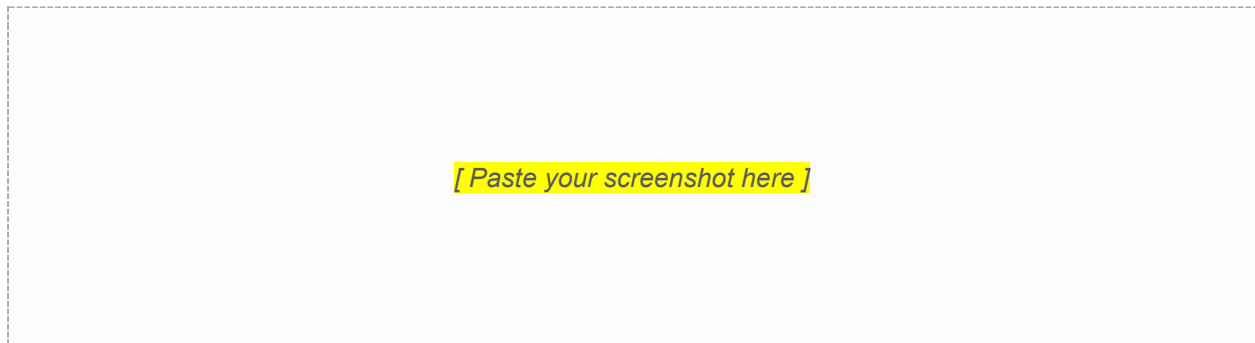


Figure 4: Alarm triggered – object closer than the threshold

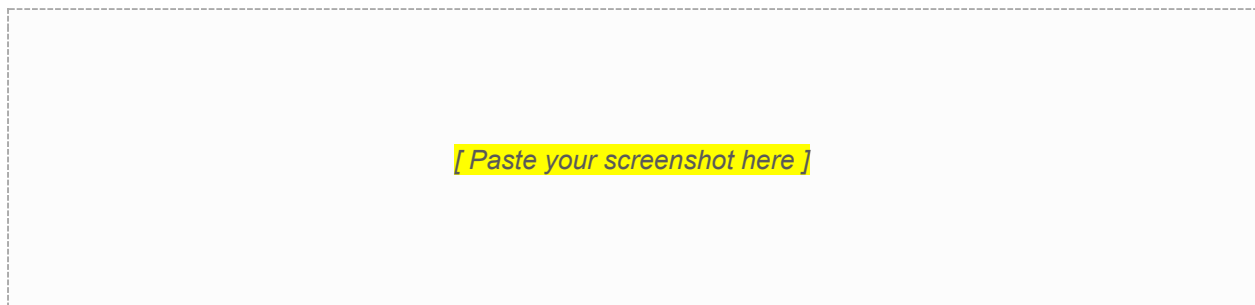


Figure 5: Distance readings in the Serial Monitor

Questions?

» How is the echo pulse width converted into a distance? Explain the / 58 step.

» What value does `pulseIn()` return if nothing is in range, and how does the `distance_cm > 0` check protect against it?

» How would you change the alarm distance, and what is the trade-off between a very small and a very large threshold?

5 Solution

All solutions will be available on [Github](#).